CampusStay: project proposal

SEZG583 Assignment 1 | Option B | TEAM DETAILS PENDING

CampusStay models a campus guesthouse booking for one room and one night. Visitors can compete for the final room, repeat a request after a lost response or encounter a payment decline. The system must preserve capacity, avoid repeated charges and recover safely from partial failure.

Four business services

Service	Responsibility	Technology	Own database
Inventory	Dated stock and holds	Python / FastAPI	campusstay_inventory
Reservation	Booking state and saga	Python / FastAPI	campusstay_reservation
Payment	Mock charges and refunds	PHP / PDO	campusstay_payment
Notification	Mock delivery and retries	PHP / PDO	campusstay_notification

Every service has its own source, configuration, dependencies, launcher and restricted database login. Services communicate over versioned REST HTTP. The optional PHP console has no database credentials. XAMPP supplies PHP/Apache and its MySQL-labelled MariaDB engine on Windows.

Transaction and assessed challenges

Reservation persists a booking, obtains an Inventory hold, charges through Payment, confirms stock and requests Notification. A decline at step 3 triggers durable compensation and releases the hold. Notification failure preserves a paid booking and can be retried.

Challenge 1 demonstrates at least 20 simultaneous attempts against capacity N with exactly N successes. Challenge 2 demonstrates sequential and simultaneous first-request idempotency, payload conflict and durable replay. The chosen extra is **3C: saga with compensation**. Timeout/outage recovery is supplementary evidence.

Deliverables and boundaries

The delivery includes code/SQL, contracts, tests/evidence, a report within 15 main pages, beginner installation/Git instructions and a timed 8-10 minute team video plan. Eight traceable rules cover date, stock, idempotency, payment before confirmation, compensation, amounts, refunds and notification state. Authentication, real providers, a full customer product and Assignment 2 containers are outside scope.

The initial implementation is substantially AI-assisted. Students must review it, make genuine contributions, complete personal reflections and record the demonstration themselves. Team identities and submission links remain pending until entered in docs/submission.json.